

Problem Solving Session

- The remainder of today's class will comprise the **problem solving session (PSS)**.
- Your instructor will divide you into **teams of 3 or 4 students**.
- Each team will **work together** to solve the following problems over the course of **20-30 minutes**.
 - You may work on paper, a white board, or digitally as determined by your instructor.
 - You will submit your solution by pushing it to GitHub before the end of class.
- Your instructor will go over the solution before the end of class.
- If there is any time remaining, you will begin work on your homework assignment.



Class participation is a significant part of your grade (20%). This includes in class activities and the problem solving session.

Your Course Assistants will grade your participation by verifying that you pushed your solutions before the end of the class period each day.

Problem Solving Team Members



Record the name of each of your problem solving team members here.

Do not forget to **add every team member's name!**
Your instructor (or course assistant) may or may not use this to determine whether or not you participated in the problem solving session.

Problem Solving 1

Modify the code on the right to turn `BinaryNode` into a generic class that can be used with any type.

Do not implement any of the methods, simply change the signatures as required.

```
public class BinaryNode<T> {  
    private T value;  
    private BinaryNode<T> left;  
    private BinaryNode<T> right;  
  
    public BinaryNode (T value) {}  
    public T getValue () {}  
    public void setValue (T value) {}  
    public BinaryNode<T> getLeft () {}  
    public void setLeft (BinaryNode<T> left) {}  
}
```

Attributes:

root
size

Methods:

toString
getSize
search
insert

Problem Solving 2

Work with your team to brainstorm attributes and methods that should be part of `BinarySearchTree` class.

Record your thoughts on the left, making sure to denote which are attributes and which are methods.

Problem Solving 3

Write the class declaration, state, and constructor for the `BinarySearchTree` you described in the previous problem.

`BinarySearchTree`s must be orderable. Does this cause problems with them being generic? Take a minute to discuss with your group and place any thoughts in a comment.

```
public class BinarySearchTree <T extends Comparable<T>>
{

    private BinaryNode <T> root;
    private int size;

    public BinarySearchTree (T initialValue) {
        root = new BinaryNode <T> (initialValue);
        size = 1;
    }
}
```

```

private boolean search (BinaryNode<E> root,
                        E target) {

    if (root.getValue () == target) {
        return true;
    }

    if (root.getLeft() != null &&
        target < root.getValue ()) {
        if (search (root.getLeft (), target) == true) {
            return true;
        }
    }
    else if (root.getRight() != null) {
        if (search (root.getRight (), target)) {
            return true;
        }
    }

    return false;
}

```

Problem Solving 4

The `binarySearch` code from Unit 5.1 has been copied to the left. The signature has been updated to work with a generic `BinarySearchTree`. Modify the implementation as necessary.

This search does not have access to the `BinaryNode`'s private data, instead it is passed in the root node. You may assume any type used for `E` will be a `Comparable`.